

A close-up photograph of a young woman with long, wavy blonde hair. She is smiling and looking off to the right. She is wearing a blue and white horizontally striped sweater. The background is blurred, showing what appears to be a brick building and some greenery.

For-løkker og tilfældighed

Program

- For-løkker
- Tilfældighed
- Øvelser
- Opsamling

A photograph of three women sitting at a white table in a modern office or cafe setting. They are all smiling and looking at laptops. The woman on the left has blonde hair and is wearing a green top. The woman in the middle has long brown hair and is wearing a purple sweater. The woman on the right has dark hair and is wearing a grey sweater. There are two laptops on the table, a blue cup, and a blue water bottle. Above them is a large planter with various green plants, including ivy and ferns. The background is a plain white wall.

for løkker

Java har to typer af løkker

- `while`: Gentag så længe en betingelse er sand
- `for`: Gentag et antal gange

Sidste gang brugte vi **while** løkker til at gentage noget,
så længe en betingelse var sand

```
while (condition) {  
    // do something  
}
```

Men fordi kun kendte **while** løkker, brugte vi dem til alt

```
int count = 0;
while (count < 10) {
    breathCycle();
    count = count + 1;
}
```

Der er en lettere måde når man ved, hvor mange gange noget skal gentages

```
for (int i = 0; i < 10; i = i + 1) {  
    breathCycle();  
}
```

```
for (int i = 0; i < 10; i = i + 1)
```

- `int i = 0` - start med at sætte en tæller `i` til 0
- `i < 10` - fortsæt så længe `i` er mindre end 10
- `i = i + 1` - øg `i` med 1 hver gang løkken kører

Hvorfor **i**, når du siger, at vi skal bruge meningsfulde navne?

i er en **konvention** i programmering, der står for "index" eller "iterator"

Ikke nødvendigt at bruge et meningsfuldt navn, da det kun bruges internt i løkken

Vi kan selvfølgelig også bruge `i++` til at øge `i` med 1

```
for (int i = 0; i < 10; i++) {  
    breathCycle();  
}
```

Tilsvarende kan vi også tælle nedad med `i--`

```
for (int i = 10; i > 0; i--) {  
    System.out.println(i);  
}  
System.out.println("Lift off!");
```

Vi kan også tælle med andre intervaller

```
for (int i = 0; i < 20; i = i + 2) {  
    System.out.println(i);  
}
```

... 2-tabellen

Vi kan også iterere hen over **char** værdier (husk at 65 er 'A', 66 er 'B', osv.)

```
for (char c = 'A'; c <= 'Z'; c++) {  
    System.out.print(c);  
}
```


A photograph of two women in an office setting. The woman on the left, wearing a white t-shirt and a watch, is looking at a laptop screen. The woman on the right, wearing a dark blazer over a white top, is looking towards the camera with a slight smile. The laptop screen shows a website with various elements. A white text box is overlaid in the center of the image.

Indlejrrede for løkker

Hvor mange 'a' 'er, 'b' 'er, osv. er der i

```
String longText = "Well, Prince, so Genoa and Lucca are ..."  
  
for (char c = 'a'; c <= 'e'; c++) {  
    for (int i = 0; i < longText.length(); i++) {  
        if (longText.charAt(i).toLowerCase() == c) {  
            System.out.print(c);  
        }  
    }  
}
```

Hvad tror i den her gør?

```
for (;;) {  
    System.out.println("Jeg kan en sang, der kan drive dig til vanvid!");  
}
```


A young woman with long, wavy blonde hair is smiling and looking off to the side. She is wearing a blue and white horizontally striped sweater. The background is a blurred outdoor setting with a brick building and some greenery. A white rectangular box is overlaid on the image, containing the word 'Tilfældighed' in a black serif font.

Tilfældighed

Vi kan bruge tilfældighed til mange ting i programmering:

- Spil (terninger, kortspil, mm.)
- Simuleringer (fysik, biologi, mm.)
- Kryptering (sikkerhed)
- Testdata (generere tilfældige data til testformål)
- Kunst (generere tilfældige mønstre, musik, mm.)

Vidste i at jeres computerspil Minecraft bruger tilfældighed til at generere verdener?

Så derfor vil vi gerne kunne generere tilfældige tal i Java

Det kan vi gøre med `java.util.Random` klassen.

Fx et tilfældigt tal **mellem 0 og 99**:

```
Random random = new Random();  
int randomNumber = random.nextInt(100);
```


... men jeg vil ha et tal **mellem 1 og 100**

```
Random random = new Random();  
int randomNumber = random.nextInt(100) + 1;
```

Hvad så med et komma-tal mellem 0.0 og 1.0?

```
Random random = new Random();  
double randomDouble = random.nextDouble();
```

Men jeg vil ha et komma-tal **mellem -1.0 og 1.0**

```
Random random = new Random();  
double randomDouble = random.nextDouble(2.0) - 1.0;
```

Det er irriterende at det er tilfældigt hver gang vi kører programmet,
jeg kan jo ikke teste det ordentligt!

Seed (frø): Vi kan sætte et startpunkt for tilfældighedsgeneratoren og få **pseudo-tilfældige** tal - der er de **samme hver gang**.

```
Random random = new Random(60); // seed  
int randomNumber = random.nextInt(100) + 1;
```

Hvilke tre ting tager du med dig fra i dag?

